Graph Representation in C

Complete C Program

```
#include <stdio.h>

#define V 6

char vertex[V] = {'A', 'B', 'C', 'D', 'E', 'F'};

/* Adjacency Matrix */
int matrix[V][V] = {0};

/* Adjacency List */
int adj[V][V];
int degree[V] = {0};

int indexOf(char ch)
{
    for (int i = 0; i < V; i++)
        if (vertex[i] == ch)
            return i;

    return -1;
}

/* Add an undirected edge */
void addEdge(int u, int v)
{
    matrix[u][v] = matrix[v][u] = 1;

    adj[u][degree[u]++] = v;
    adj[v][degree[v]++] = u;
}

/* BFS using Adjacency Matrix */
void BFSMatrix(int start)
{
    int visited[V] = {0};
    int queue[V], front = 0, rear = 0;

    visited[start] = 1;
    queue[rear++] = start;

    printf("BFS (Matrix): ");

    while (front < rear)
    {
        int u = queue[front++];
        printf("%c ", vertex[u]);

        for (int v = 0; v < V; v++)
        {
            if (matrix[u][v] && !visited[v])
            {
                visited[v] = 1;
                queue[rear++] = v;
            }
        }
    }

    printf("\\n");
}

/* DFS using Adjacency Matrix */
void DFSMatrixUtil(int u, int visited[])
{
    visited[u] = 1;
    printf("%c ", vertex[u]);

    for (int v = 0; v < V; v++)
    {
        if (matrix[u][v] && !visited[v])
            DFSMatrixUtil(v, visited);
    }
}

void DFSMatrix(int start)
{
    int visited[V] = {0};

    printf("DFS (Matrix): ");
    DFSMatrixUtil(start, visited);
    printf("\\n");
}

/* BFS using Adjacency List */
void BFSList(int start)
```

```
{
    int visited[V] = {0};
    int queue[V], front = 0, rear = 0;

    visited[start] = 1;
    queue[rear++] = start;

    printf("BFS (List): ");

    while (front < rear)
    {
        int u = queue[front++];
        printf("%c ", vertex[u]);

        for (int i = 0; i < degree[u]; i++)
        {
            int v = adj[u][i];

            if (!visited[v])
            {
                visited[v] = 1;
                queue[rear++] = v;
            }
        }
    }

    printf("\\n");
}

/* DFS using Adjacency List */
void DFSListUtil(int u, int visited[])
{
    visited[u] = 1;
    printf("%c ", vertex[u]);

    for (int i = 0; i < degree[u]; i++)
    {
        int v = adj[u][i];

        if (!visited[v])
            DFSListUtil(v, visited);
    }
}

void DFSList(int start)
{
    int visited[V] = {0};

    printf("DFS (List): ");
    DFSListUtil(start, visited);
    printf("\\n");
}

/* Search using BFS and count operations */
int searchMatrix(char target, int *visits, int *checks)
{
    int start = indexOf('A');
    int t = indexOf(target);
    int visited[V] = {0};
    int queue[V], front = 0, rear = 0;

    *visits = *checks = 0;

    if (t == -1)
        return 0;

    visited[start] = 1;
    queue[rear++] = start;

    while (front < rear)
    {
        int u = queue[front++];
        (*visits)++;

        if (u == t)
            return 1;

        for (int v = 0; v < V; v++)
        {
            (*checks)++;

            if (matrix[u][v] && !visited[v])
            {
                visited[v] = 1;
                queue[rear++] = v;
            }
        }
    }

    return 0;
```

```
}

int searchList(char target, int *visits, int *checks)
{
    int start = indexOf('A');
    int t = indexOf(target);
    int visited[V] = {0};
    int queue[V], front = 0, rear = 0;

    *visits = *checks = 0;

    if (t == -1)
        return 0;

    visited[start] = 1;
    queue[rear++] = start;

    while (front < rear)
    {
        int u = queue[front++];
        (*visits)++;

        if (u == t)
            return 1;

        for (int i = 0; i < degree[u]; i++)
        {
            (*checks)++;
            int v = adj[u][i];

            if (!visited[v])
            {
                visited[v] = 1;
                queue[rear++] = v;
            }
        }
    }

    return 0;
}

int main()
{
    char target;
    int vm, cm, vl, cl;

    /* Construct the graph */
    addEdge(0, 1);  /* A-B */
    addEdge(0, 2);  /* A-C */
    addEdge(1, 3);  /* B-D */
    addEdge(1, 4);  /* B-E */
    addEdge(2, 5);  /* C-F */
    addEdge(4, 5);  /* E-F */

    printf("ADJACENCY MATRIX\\n");
    printf("  A B C D E F\\n");

    for (int i = 0; i < V; i++)
    {
        printf("%c ", vertex[i]);

        for (int j = 0; j < V; j++)
            printf("%d ", matrix[i][j]);

        printf("\\n");
    }

    printf("\\nADJACENCY LIST\\n");

    for (int i = 0; i < V; i++)
    {
        printf("%c: ", vertex[i]);

        for (int j = 0; j < degree[i]; j++)
            printf("%c ", vertex[adj[i][j]]);

        printf("\\n");
    }

    printf("\\nGRAPH TRAVERSAL\\n");
    BFSMatrix(0);
    DFSMatrix(0);
    BFSList(0);
    DFSList(0);

    printf("\\nEnter vertex to search from A: ");
    scanf(" %c", &target);

    if (searchMatrix(target, &vm, &cm))
        printf("\\nMatrix: Vertex %c found.\\n", target);
```

```
    else
        printf("\\nMatrix: Vertex %c not found.\\n", target);

    printf("Vertices examined: %d\\n", vm);
    printf("Matrix entries checked: %d\\n", cm);

    if (searchList(target, &vl, &cl))
        printf("\\nList: Vertex %c found.\\n", target);
    else
        printf("\\nList: Vertex %c not found.\\n", target);

    printf("Vertices examined: %d\\n", vl);
    printf("List entries checked: %d\\n", cl);

    return 0;
}
```

Possible Output

Input: E

```
ADJACENCY MATRIX
  A B C D E F
A 0 1 1 0 0 0
B 1 0 0 1 1 0
C 1 0 0 0 0 1
D 0 1 0 0 0 0
E 0 1 0 0 0 1
F 0 0 1 0 1 0

ADJACENCY LIST
A: B C
B: A D E
C: A F
D: B
E: B F
F: C E

GRAPH TRAVERSAL
BFS (Matrix): A B C D E F
DFS (Matrix): A B D E F C
BFS (List): A B C D E F
DFS (List): A B D E F C

Enter vertex to search from A: E

Matrix: Vertex E found.
Vertices examined: 5
Matrix entries checked: 24

List: Vertex E found.
Vertices examined: 5
List entries checked: 8
```